CSE 21 Fall 2026
Homework 5
Due date: Monday Nov 9, 2026 at 11:59pm

Instructions

This homework should be done in **groups** of one to four students, without assistance from anyone besides the instructional staff and your group members. Homework must be submitted through Gradescope by a **single representative** of your group and received by **11:59pm** on the due date. There are no exceptions to this rule.

You will be able to look at your scanned work before submitting it. You must **type** your solutions. (hand-drawn diagrams are okay.) Your group representative can resubmit your assignment as many times as you like before the deadline. Only the most recent submission will be graded.

Students should consult their textbook, class notes, lecture slides, podcasts, group members, instructors, TAs, and tutors when they need help with homework. You may ask questions about the homework in office hours, but questions on Piazza should be private, visible only to instructors.
This assignment will be graded for not only the *correctness* of your answers, but on your ability to present your ideas clearly and logically. You should explain or justify, present clearly how you arrived at your conclusions and justify the correctness of your answers with mathematically sound reasoning (unless explicitly told not to). Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to **convince the reader** that your results and methods are sound.
Key Concepts Sorting, searching, asymptotic notation, time analysis.

---

1. (12 points) Let $f : \mathbb{R} \to \mathbb{R}$ be a continuous increasing function. Then we can use a version of binary search to solve for the "root" of the function f (value x such that $f(x) = 0$.) This algorithm will take the input f, starting value n such that $0 < r < n$ where r is the root of the function and an error term e.

   The function will stop when it finds a value v such that $|r - v| \leq e$.

   FunctionSolver(f, n, e)

   1. $lo = 0$
   2. $hi = n$
   3. $v = (hi + lo)/2$
   4. **while** $hi - lo > 2e$:
   5. **if** $f(v) = 0$:
   6. **return** v
   7. **if** $f(v) < 0$:
   8. $lo = v$
   9. **if** $f(v) > 0$:
   10. $hi = v$
   11. $v = (hi + lo)/2$
   12. **return** v

   (a) Trace through the algorithm on the input *FunctionSolver*$(x^2 - x - 1, 4, 0.0625)$.
   (you can handdraw the table)
   (Here is an example tracethrough of *FunctionSolver*$(e^x - 1/x, 2, 0.0625)$:)

iteration	$hi - lo > 2e$	$f(v)$	$f(v)$__0	lo	hi	v
0				0	2	1
1	TRUE	1.72	$>$	0	1	0.5
2	TRUE	-0.35	$<$	0.5	1	0.75
3	TRUE	0.78	$>$	0.5	0.75	0.625
4	TRUE	0.27	$>$	0.5	0.625	0.5625
5	FALSE					

   return 0.5625

   (b) A loop invariant for this algorithm is:

   After each iteration, $lo \leq r \leq hi$.

   Prove this loop invariant.

Solution: (a) Table
Given $f(x) = x^2 - x - 1$, $n = 4$, and $e = 0.0625$.

The stopping condition is $hi - lo \leq 0.125$.

Iter.	$hi - lo > 2e$	$f(v)$	lo	hi	v
0	$-$	$-$	0	4	2
1	T	1	0	2	1
2	T	-1	1	2	1.5
3	T	-0.25	1.5	2	1.75
4	T	0.3125	1.5	1.75	1.625
5	T	0.015625	1.5	1.625	1.5625
6	F	$-$	1.5	1.625	1.5625

Since $hi - lo = 0.125 = 2e$, the loop terminates.

$$\boxed{v = 1.5625}$$

(b) Loop Invariant Proof

We prove by induction that $lo \leq r \leq hi$.

Base Case:

Initially, $lo = 0$ and $hi = n$. Since we are given $0 < r < n$, we have

$$lo \leq r \leq hi.$$

Thus, the invariant holds initially.

Inductive Hypothesis:

Assume $lo \leq r \leq hi$ holds at the beginning of iteration t.

Inductive Step:

The algorithm computes $v = (hi + lo)/2$.

Since f is increasing and $f(r) = 0$, we consider three cases.

If $f(v) < 0$, then $v < r$. The algorithm sets $lo = v$, so

$$lo_{\text{new}} = v < r \leq hi.$$

If $f(v) > 0$, then $v > r$. The algorithm sets $hi = v$, so

$$lo \leq r < v = hi_{\text{new}}.$$

If $f(v) = 0$, the algorithm returns the exact root and terminates.

Therefore, the invariant holds after iteration t, and hence at iteration $t + 1$.

Conclusion:

By induction, $lo \leq r \leq hi$ is maintained throughout the algorithm.

2. (15 points) Consider the following sorting algorithm that takes a list of integers as an input and outputs a sorted list of those elements. (note, the MERGE subroutine is an algorithm we will talk about on Friday 11/7. All you need to know about MERGE is that it takes two sorted lists of integers and returns a sorted list of all the integers combined. If the size of the lists are j and k, then the runtime of MERGE is $O(k + j)$.)

QueueSort$(a_1, \ldots, a_n)$

1. Initialize a queue Q of lists so that each element of the input is in a list all by itself.
2. $Q = ([a_1], [a_2], \ldots, [a_n])$
3. **while** $|Q| \geq 2$:
4. dequeue the first two lists from Q and MERGE the two lists together.
5. enqueue the result into Q.
6. **return** Q

Example: Sort the list: (10,8,3,2,8,7,6)

$$
\begin{aligned}
& & ([10],[8],[3],[2],[8],[7],[6]) \\
MERGE([10],[8]) \leftarrow & & ([3],[2],[8],[7],[6]) \\
& & ([3],[2],[8],[7],[6]) \leftarrow [8,10] \\
& & ([3],[2],[8],[7],[6],[8,10]) \\
MERGE([3],[2]) \leftarrow & & ([8],[7],[6],[8,10]) \\
& & ([8],[7],[6],[8,10]) \leftarrow [2,3] \\
& & ([8],[7],[6],[8,10],[2,3]) \\
MERGE([8],[7]) \leftarrow & & ([6],[8,10],[2,3]) \\
& & ([6],[8,10],[2,3]) \leftarrow [7,8] \\
& & ([6],[8,10],[2,3],[7,8]) \\
MERGE([6],[8,10]) \leftarrow & & ([2,3],[7,8]) \\
& & ([2,3],[7,8]) \leftarrow [6,8,10] \\
& & ([2,3],[7,8],[6,8,10]) \\
MERGE([2,3],[7,8]) \leftarrow & & ([6,8,10]) \\
& & ([6,8,10]) \leftarrow [2,3,7,8] \\
& & ([6,8,10],[2,3,7,8]) \\
MERGE([6,8,10],[2,3,7,8]) \leftarrow & & (\ ) \\
& & (\ ) \leftarrow [2,3,6,7,8,8,10] \\
& & ([2,3,6,7,8,8,10])
\end{aligned}
$$

Consider the loop invariant:

After each iteration, every list in Q is sorted.

(a) (6 points) Prove this loop invariant using induction.

(b) (3 points) Use the loop invariant to show that the algorithm is correct.

(c) (3 points) Use the runtime method we learned in class (product rule for loops) to show that this algorithm runs in $O(n^2)$ time.

(d) (3 points) Show that $O(n^2)$ is not a tight bound by doing a more careful analysis.

Solution: (a)
We prove by induction that every list in Q is sorted.

Base Case:

Initially, $Q = ([a_1],[a_2],\ldots,[a_n])$. Each list contains only one element, so every list is sorted. Thus, the invariant holds initially.

Inductive Hypothesis:

Assume that after iteration t, every list in Q is sorted.

Inductive Step:

At iteration $t+1$, the algorithm removes the first two lists from Q. By the inductive hypothesis, both lists are sorted.

The MERGE function combines two sorted lists into one sorted list, which is then added back to Q.

All remaining lists are unchanged and sorted. Therefore, every list in Q is still sorted.

Conclusion:

By induction, the loop invariant holds after every iteration.

(b)

Initially, Q contains n lists. Each iteration removes two lists and adds one, decreasing the number of lists by one.

Thus, for $n \geq 1$, the loop terminates with exactly one list in Q.

By the loop invariant, this remaining list is sorted.

Also, MERGE preserves all elements from both lists, so no elements are lost or added.

Therefore, the final list contains exactly the original elements in sorted order, proving that the algorithm is correct.

(c)

Each iteration decreases the number of lists by one. Therefore, the loop executes $n - 1 = O(n)$ times.

MERGE takes $O(k + j)$ time, where k and j are the sizes of the two lists being merged.

Since there are only n elements in total,

$$k + j \leq n.$$

Thus, each iteration takes at most $O(n)$ time.

Using the product rule for loops,

$$T(n) = O(n) \cdot O(n) = \boxed{O(n^2)}.$$

(d)

The $O(n^2)$ bound is not tight because not every merge involves n elements.

Since the algorithm merges lists in queue order, the lists being merged have roughly equal sizes.

In the first round, merging lists of size 1 costs $O(2)$ per merge, with about $n/2$ merges:

$$\frac{n}{2} \cdot O(2) = O(n).$$

In the next round, merging lists of size 2 costs $O(4)$ per merge, with about $n/4$ merges:

$$\frac{n}{4} \cdot O(4) = O(n).$$

This continues, with each round taking $O(n)$ time.

Since the list sizes approximately double each round, there are $O(\log n)$ rounds.

Therefore,

$$T(n) = O(n) \cdot O(\log n) = \boxed{O(n \log n)}.$$

Thus, $O(n^2)$ is not a tight upper bound.

3. (12 points) For each algorithm, compute the exact number of times the algorithm prints in terms of n and express this expression in terms of Θ (in simplest form), where $n \geq 3$. (We use the convention that if there is a for loop such that the starting value is greater than the ending value, then that loop does not run.)

(a)
```
for i = 1 to n:
.    for j = 1 to n + i:
.        print (i, j).
```

(b)
```
for i = 1 to n:
.    for j = 2i to 2n:
.        print (i, j).
```

(c)
```
for i = 1 to n − 2:
.    for j = i + 1 to n − 1:
.        for k = j + 1 to n:
.            print (i, j, k)
```

(a)

For each i, the inner loop runs $n + i$ times.

$$T(n) = \sum_{i=1}^{n} (n + i)$$

$$= n^2 + \frac{n(n+1)}{2} = \boxed{\frac{3n^2 + n}{2}}$$

Therefore,

$$\boxed{\Theta(n^2)}$$

(b)

For each i, the inner loop runs $2n - 2i + 1$ times.

$$T(n) = \sum_{i=1}^{n} (2n - 2i + 1)$$

$$= n(2n + 1) - 2\frac{n(n+1)}{2}$$

$$= \boxed{n^2}$$

Therefore,

$$\boxed{\Theta(n^2)}$$

(c)

The loops generate all triples satisfying $1 \leq i < j < k \leq n$.

Each triple corresponds to choosing 3 distinct indices from n indices.

Therefore,

$$T(n) = \binom{n}{3}$$

$$= \boxed{\frac{n(n-1)(n-2)}{6}}$$

Since the highest-degree term is n^3,

$$\boxed{\Theta(n^3)}$$